Федеральное государственное автономное образовательное учреждение высшего образования
«РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ ИМЕНИ ПАТРИСА ЛУМУМБЫ»
(РУДН)

Основное учебное подразделение факультет физико-математических и естественных наук

Направление/специальность 02.03.01 «Математика и компьютерные науки», специализация «Математика и компьютерные науки»

ОТЧЕТ

о прохождении преддипломной практики

(вид и наименование практики)

Петров Артем Евгеньевич

(Ф.И.О. обучающегося)

Курс, группа 4, НКНбд-01-22

Место прохождения практики Научный центр вычислительных методов и моделирования киберфизических систем РУДН

полное наименование организации (предприятия)

Сроки прохождения с 06.04.2026 г. по 30.04.2026 г.

Руководители практики:

от РУДН Кройтор О.К., асс. каф. ММиИИ
Ф.И.О., должность

от организации (предприятия)
проф. Самуйлов К.Е., директор ИКНиТ
Ф.И.О., должность

Оценка_____

Москва 2026 г.

Оглавление

Список сокращений 3

Список основных обозначений 4

Введение 5

Глава 1. Теоретические основы 7

1.1. Рынок деривативов и задача оценки опционов 7

1.2. Модель Блэка–Шоулса 7

1.3. Биномиальная модель Кокса–Росса–Рубинштейна 8

1.4. Нейросетевая эквивалентность BTNet: теоремы Шорохова С.Г. 9

Глава 2. Программная реализация и эксперименты 9

2.1. Интеграция библиотеки QuantLib с программным комплексом BTNet 10

2.1.1. Описание программного комплекса BTNet 10

2.1.2. Настройка среды и подключение библиотеки QuantLib 12

2.1.3. Реализация функций прайсинга опционов 12

2.1.4. Сравнительный анализ точности методов 13

2.2. Вычисление греческих символов опционов через автоматическое дифференцирование 14

2.2.1. Реализация греческих символов европейского пут-опциона 14

2.2.2. Эффект Gamma = 0 у кусочно-линейных нейронных сетей 15

2.2.3. Реализация греческих символов американского пут-опциона 16

2.2.4. Эксперимент с переносом весов European → American 17

Заключение 18

Список источников 21

Приложение А. Исходный код пакета btnn_bs 22

A.1. Базовые слои (layers.py) 22

A.2. Архитектура BTNetEuropean (model_european.py) 24

A.3. Архитектура BTNetAmerican (model_american.py) 24

A.4. Функция обучения train_BTNet (training.py) 25

A.5. Вычисление греческих символов через autograd (greeks.py) 26

Список сокращений

Русскоязычные сокращения

ВКР — выпускная квалификационная работа

ПК — программный комплекс

БШ — Блэк-Шоулз

Англоязычные сокращения

API — Application Programming Interface (интерфейс прикладного программирования)

BAW — Barone-Adesi-Whaley (аналитическое приближение для американских опционов)

BS — Black-Scholes (модель ценообразования опционов)

CRR — Cox-Ross-Rubinstein (биномиальная модель ценообразования)

MAE — Mean Absolute Error (средняя абсолютная ошибка)

RMSE — Root Mean Square Error (среднеквадратичная ошибка)

Список основных обозначений

S_0 — текущая цена базового актива

K — цена исполнения опциона (страйк)

T — время до экспирации, лет

r — безрисковая процентная ставка

σ — волатильность доходности базового актива

V(S, t) — цена опциона при цене актива S в момент времени t

Δ (Delta) — производная цены опциона по цене базового актива, dV/dS_0

Γ (Gamma) — вторая производная цены по S_0, d^2V/dS_0^2

ν (Vega) — производная цены опциона по волатильности, dV/dσ

Θ (Theta) — производная цены опциона по времени, –dV/dT

u, d — коэффициенты роста и убывания цены в биномиальной модели CRR

p — риск-нейтральная вероятность роста цены актива

n — число временных шагов биномиального дерева

Δt — длина одного временного шага, Δt = T/n

W — весовой фильтр нейросетевой архитектуры BTNet

Введение

Апробация: результаты работы представлены на предзащите ВКР кафедры теории вероятностей и математической статистики РУДН (апрель 2026 г.).

Структура работы: ВКР состоит из введения, двух глав, заключения, списка из 14 источников и приложений. Глава 1 посвящена теоретическим основам моделирования опционов. Глава 2 описывает программную реализацию и результаты вычислительных экспериментов.

Практическая значимость: реализованный пакет btnn_bs позволяет оценивать американские пут-опционы и вычислять их греческие символы посредством автоматического дифференцирования, что может быть использовано в задачах управления рыночным риском.

Научная новизна: в работе впервые реализован и верифицирован программный пакет btnn_bs на основе теоретических результатов Шорохова С.Г. (2024); получен и обоснован эффект Gamma = 0 как фундаментальное ограничение ReLU-архитектур при вычислении вторых производных.

Предмет исследования: архитектура нейронной сети BTNet, аналитическая инициализация её весов и автоматическое дифференцирование для вычисления греческих символов опционов.

Объект исследования: американский пут-опцион и методы его численной оценки.

Для достижения цели поставлены следующие задачи: (1) изучить теоретические основы биномиальной модели CRR и нейросетевой эквивалентности BTNet; (2) реализовать архитектуры BTNetEuropean и BTNetAmerican в фреймворке PyTorch; (3) верифицировать точность прайсинга относительно эталона QuantLib; (4) вычислить греческие символы опционов через автоматическое дифференцирование и исследовать их свойства; (5) провести эксперимент с переносом весов и оценить его влияние на точность греческих символов.

Цель работы: реализовать и верифицировать нейросетевую архитектуру BTNet, прямой проход которой математически эквивалентен обратной индукции по биномиальному дереву CRR, и исследовать вычисление греческих символов опционов через автоматическое дифференцирование как инструмент оценки рыночного риска. [1]

Задача оценки американского пут-опциона относится к классу задач оптимальной остановки с подвижной границей раннего исполнения, для которой не существует аналитического решения в замкнутой форме. Стандартным численным методом является биномиальное дерево Кокса–Росса–Рубинштейна (CRR), решающее задачу обратной индукцией. В работе Шорохова С.Г. (2024) доказано, что прямой проход нейронной сети специальной архитектуры (BTNet) математически эквива-

лентен этой индукции. Настоящая выпускная квалификационная работа посвящена программной реализации и верификации данной архитектуры, а также исследованию возможности вычисления греческих символов опционов (Delta, Gamma, Vega, Theta) через механизм автоматического дифференцирования PyTorch. [1; 2]

Актуальность: производные финансовые инструменты (деривативы) — ключевой элемент современных финансовых рынков. Точная и быстрая оценка стоимости опционов и их чувствительностей к рыночным параметрам (греческих символов) необходима для управления рыночным риском. Нейросетевые методы открывают возможность получать греческие символы без дополнительных численных схем — через автоматическое дифференцирование. [4; 7]

Глава 1. Теоретические основы

1.1. Рынок деривативов и задача оценки опционов

Опцион — это финансовый контракт, дающий его покупателю право (но не обязанность) купить (колл) или продать (пут) базовый актив по фиксированной цене исполнения K в момент времени T (европейский) или в любой момент до T (американский). Рынок деривативов достигает сотен триллионов долларов в номинальном выражении, а точная оценка стоимости опционов критически важна для управления рыночным риском. Функция выплаты американского пут-опциона: $h(S) = \max(K - S, 0)$. Задача оценки сводится к нахождению стоимости $V(S_0, t)$ при текущей цене актива S_0 и времени до экспирации $T - t$. [4]

Ключевое различие между европейским и американским опционами состоит в праве досрочного исполнения. Для пут-опциона это право имеет положительную ценность («премия за раннее исполнение»), поэтому цена американского пута $P_am(S, t)$ всегда не ниже цены европейского $P_eu(S, t)$ при тех же параметрах. Разность $P_am - P_eu$ называется ранней премией исполнения и не имеет аналитического выражения в рамках стандартной модели Блэка–Шоулса.

Математически задача оценки американского пут-опциона формулируется как задача оптимальной остановки: держатель выбирает момент времени $\tau \in [0, T]$ для исполнения опциона, максимизируя ожидаемую выплату $\max(K - S\tau, 0) \cdot e^{(-r\tau)}$. Существование оптимальной стратегии остановки эквивалентно существованию «подвижной краевой границы» — критического уровня цены актива $S^*(t)$, ниже которого немедленное исполнение оптимально. Определение $S^*(t)$ аналитически не представляется возможным, что и обусловливает необходимость численных методов.

На финансовых рынках деривативы (производные инструменты) — это контракты, стоимость которых определяется ценой базового актива: акции, индекса, валюты или сырьевого товара. Опционы относятся к наиболее распространённому классу деривативов и широко применяются для хеджирования рисков, арбитража и спекулятивных стратегий.

1.2. Модель Блэка–Шоулса

Модель Блэка–Шоулса (1973) описывает динамику цены базового актива геометрическим броуновским движением: $dS = \mu S\, dt + \sigma S\, dW$, где μ — ожидаемая доходность, σ — волатильность, W — стандартный брауновский процесс. В условиях безарбитражности стоимость европейского колл-опциона задаётся формулой: $C = S_0 N(d_1) - Ke^{-rT} N(d_2)$, где $d_1 = (\ln(S_0/K) + (r + \sigma^2/2)T)/(\sigma\sqrt{T})$, $d_2 = d_1 - \sigma\sqrt{T}$, $N(\cdot)$ — функция нормального распределения. Для американских опционов аналитическое решение недостижимо; применяются численные методы, в том числе приближение Бароне–Адези–Уэйли. [3; 6]

Модель строится на ряде допущений: непрерывность торгов, отсутствие транзакционных издержек, постоянство σ и r, отсутствие дивидендов. Нарушение этих условий на практике ведёт к понятию «подразумеваемой волатильности» — значения σ, при котором формула БШ воспроизводит рыночную цену. Для американских опционов формула БШ неприменима, поскольку не учитывает досрочное исполнение.

Аналитическая формула цены европейского пут-опциона в модели БШ: $P_eu = K \cdot e^{(-rT)} \cdot N(-d_2) - S_0 \cdot N(-d_1)$, где $d_1 = [\ln(S_0/K) + (r + \sigma^2/2)T] / (\sigma\sqrt{T})$, $d_2 = d_1 - \sigma\sqrt{T}$, $N(\cdot)$ — функция нормального распределения. Формула получена из безарбитражного аргумента (уравнение БШ) при граничном условии $P(S, T) = \max(K - S, 0)$.

В рамках модели Блэка–Шоулса цена актива S подчиняется стохастическому дифференциальному уравнению (геометрическое броуновское движение): $dS = \mu S\, dt + \sigma S\, dW$, где μ — ожидаемая доходность, σ — мгновенная волатильность, W — стандартный процесс Винера. В риск-нейтральном мире μ заменяется на безрисковую ставку r.

1.3. Биномиальная модель Кокса–Росса–Рубинштейна

Биномиальная модель CRR (Cox, Ross, Rubinstein, 1979) дискретизирует динамику цены актива на n временных шагов $\Delta t = T/n$. На каждом шаге цена умножается на u (рост) или d (падение). Риск-нейтральная вероятность роста: $p = (e^{r\Delta t} - d)/(u - d)$. Стоимость европейского опциона вычисляется прямой суммой по листьям дерева, взвешенной биномиальными вероятностями. Для американского опциона применяется обратная индукция: на каждом узле стоимость опциона равна максимуму из продолженной стоимости (дисконтированное ожидание) и стоимости немедленного исполнения. Это условие определяет подвижную границу раннего исполнения. Наряду с биномиальной моделью применяются методы Монте-Карло [8], в частности метод наименьших квадратов Лонгстаффа–Шварца [9]. [2; 5]

Погрешность метода CRR при n шагах составляет $O(1/n)$ по цене и уменьшается при увеличении числа шагов. На практике $n = 500$ достаточно для достижения точности, сопоставимой с аналитическими приближениями Barone–Adesi–Whaley. Именно значение $n = 500$ используется как эталон в настоящей работе.

Алгоритм обратной индукции для американского пут-опциона строится следующим образом. Сначала вычисляется цена актива во всех узлах конечного слоя: $S(i, j) = S_0 \cdot u^j \cdot d^{(i-j)}$, где i — номер шага, j — число «роста». Затем инициализируется граничное условие: $V(n, j) = \max(K - S(n, j), 0)$. Далее для каждого шага i от $n-1$ до 0 применяется рекуррентное соотношение: $V(i, j) = \max(K - S(i, j), e^{(-r\Delta t)} \cdot (p \cdot V(i+1, j+1) + q \cdot V(i+1, j)))$. Первое слагаемое — стоимость немедленного исполнения, второе — дисконтированная стоимость продолжения.

Параметры CRR вычисляются из волатильности и шага времени $\Delta t = T/n$: $u = \exp(\sigma\sqrt{\Delta t})$ — коэффициент роста; $d = 1/u$ — коэффициент падения; риск-

нейтральная вероятность роста $p = (\exp(r\Delta t) - d)/(u - d)$, вероятность падения $q = 1 - p$. При $n \to \infty$ биномиальное распределение сходится к логнормальному, и цены CRR стремятся к ценам БШ.

1.4. Нейросетевая эквивалентность BTNet: теоремы Шорохова С.Г.

Шорохов С.Г. (2024) доказал, что прямой проход нейронной сети специальной архитектуры математически эквивалентен обратной индукции по биномиальному дереву. Архитектура BTNetEuropean включает DenseLayer (аналог инициализации листьев) и n последовательных ConvLayer с активацией ReLU, воспроизводящих свёртку для европейского пут-опциона. Архитектура BTNetAmerican заменяет ConvLayer на MaxoutLayer, реализуя операцию max(продолженная стоимость, стоимость исполнения), соответствующую условию раннего исполнения. [1] Применение нейронных сетей для оценки производных финансовых инструментов исследовалось ранее в работе [7].

Аналитическая инициализация весов производится из параметров CRR: $u = \exp(\sigma\sqrt{\Delta t})$, $d = 1/u$, $p = (\exp(r\Delta t) - d)/(u - d)$, $q = 1 - p$. Веса фильтра задаются как $w = [p \cdot e^{-r\Delta t}, q \cdot e^{-r\Delta t}]$. Ключевое следствие: дифференцируемость сети по входным параметрам (S_0, σ, r, T) позволяет вычислять греческие символы опционов через torch.autograd.grad без дополнительных численных схем. [1; 11]

Таким образом, при аналитической инициализации весов архитектура BTNet является детерминированной: обучение не требуется, и модель сразу даёт численно точные результаты. Это позволяет использовать PyTorch autograd для вычисления производных (греков) без дополнительного обучения или численных конечных разностей.

DenseLayer преобразует вектор начальных цен актива в граничные условия: он вычисляет $\max(K - S[j], 0)$ для каждого узла на конечном временном слое. Это соответствует терминальному граничному условию опционного контракта. После DenseLayer применяются n слоёв ConvLayer (или MaxoutLayer), каждый из которых уменьшает размерность вектора на 1 — ровно так же, как обратная индукция сворачивает биномиальное дерево от n+1 узлов к 1.

Слои ConvLayer реализуют дискретную свёртку с окном размера 2 и одним фильтром, что соответствует операции взвешенного суммирования двух соседних элементов: $y[j] = w_0 \cdot x[j+1] + w_1 \cdot x[j]$. При инициализации $w = [p \cdot e^{(-r\Delta t)}, q \cdot e^{(-r\Delta t)}]$ и применении ReLU-активации к разности с payoff каждый ConvLayer воспроизводит один шаг обратной индукции. В BTNetAmerican операция ReLU заменена на maxout: $y[j] = \max(w_0 \cdot x[j+1] + w_1 \cdot x[j], K - S[j])$, что реализует выбор между продолжением и исполнением.

Глава 2. Программная реализация и эксперименты

2.1. Интеграция библиотеки QuantLib с программным комплексом BTNet

2.1.1. Описание программного комплекса BTNet

Объектом исследования в рамках преддипломной практики является программный комплекс BTNet — нейросетевая архитектура, математически эквивалентная биномиальному дереву Кокса–Росса–Рубинштейна (CRR). [2] Эквивалентность установлена теоремами Шорохова (2024). [1]

Теорема 1 (европейский пут): прямой проход сети BTNetEuropean, состоящей из одного плотного слоя (DenseLayer) и n свёрточных слоёв (ConvLayer), при аналитической инициализации весов точно воспроизводит прямую индукцию CRR-дерева для европейского пут-опциона. [1]

Теорема 2 (американский пут): прямой проход сети BTNetAmerican, содержащей DenseLayer и n слоёв с активациями maxout, при аналитической инициализации точно воспроизводит обратную индукцию с учётом условия раннего исполнения. [1]

Веса аналитически инициализируются из параметров CRR: коэффициенты $u = \exp(\sigma\sqrt{\Delta t})$, $d = 1/u$ и риск-нейтральная вероятность $p = (\exp(r\Delta t) - d)/(u - d)$, где $\Delta t = T/n$. В экспериментах использовалась архитектура с $n = 9$ слоями; параметры тестовой задачи: $S_0 = 0.5$, $\sigma = 0.25$, $r = 0.05$, $T = 1$ (обучающая выборка — 200 точек, тестовая сетка — 100 точек). Обучение проводилось методом Adam (lr = 0.005, 500 эпох) с минимизацией среднеквадратичной ошибки на синтетической выборке страйков. Дообучение модели выполняется оптимизатором Adam [14].

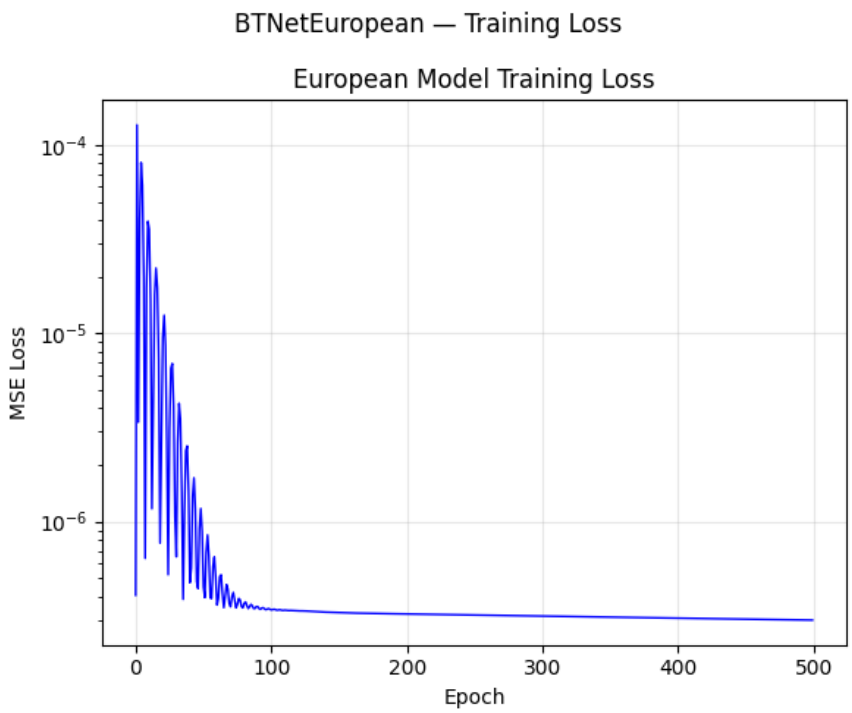

Рисунок 1 — Динамика функции потерь BTNetEuropean в процессе обучения (500 эпох)

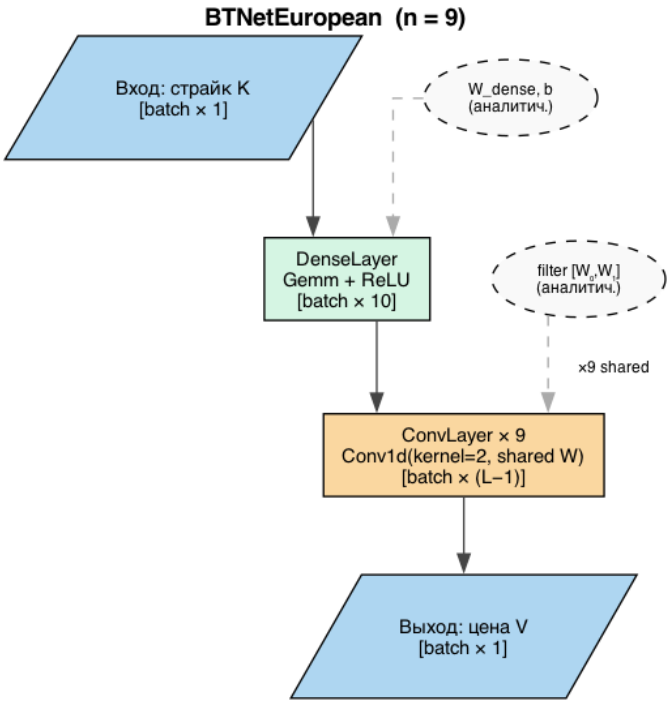

Рисунок A.1 — Архитектура BTNetEuropean (DenseLayer + 9 × ConvLayer)

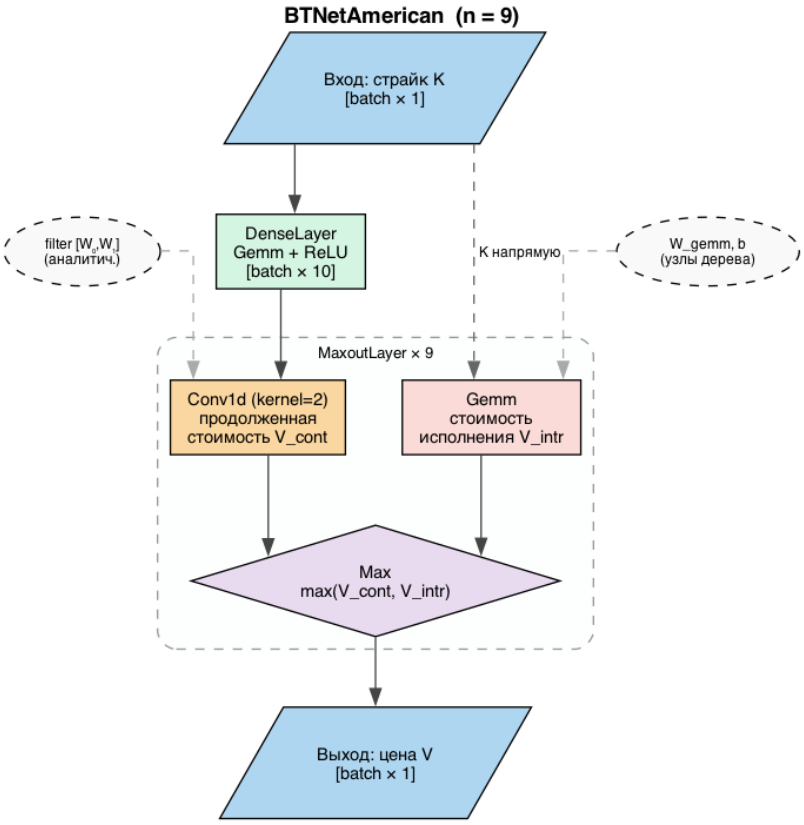

Рисунок А.2 — Архитектура BTNetAmerican (DenseLayer + 9 × MaxoutLayer)

2.1.2. Настройка среды и подключение библиотеки QuantLib

— SimpleQuote — объект котирования рыночного параметра, поддерживающий динамическое обновление значения;

— FlatForward — плоская кривая безрисковой доходности;

— BlackScholesMertonProcess — стохастический процесс БШМ, объединяющий цену актива, дивидендную доходность, безрисковую ставку и волатильность.

Реализованный модуль интеграции — btnn_bs/quantlib.py. [10]

Настройка среды выполнения включает установку глобальной расчётной даты (Settings.instance().evaluationDate) и создание вспомогательных объектов для описания рыночной конъюнктуры. Ключевые компоненты:

QuantLib — открытая C++ библиотека для количественных финансов, предоставляющая реализации широкого спектра моделей ценообразования деривативов. Python-обёртка QuantLib-Python (SWIG) позволяет использовать её в среде PyTorch без потери производительности. В данной работе QuantLib выступает источником эталонных цен: результаты BTNet сравниваются с ценами, полученными методами QuantLib, для оценки точности нейросетевого подхода.

2.1.3. Реализация функций прайсинга опционов

Для бенчмаркинга результатов BTNet реализованы три функции вычисления цен на сетке из 100 значений S_0 при фиксированных $K = 1.0$, $\sigma = 0.25$, $r = 0.05$, $T = 1$:

1. european_put_grid — цены европейского пута через AnalyticEuropeanEngine (замкнутая формула Блэка–Шоулса). [3] Результаты верифицированы относительно аналитических BS-формул.

2. american_put_grid_crr — цены американского пута через BinomialVanillaEngine с CRR-схемой (500 шагов). Анализ сходимости показал устойчивость результата при steps ≥ 300; значение при steps = 500 принято в качестве эталона. [2]

3. american_put_grid_baw — цены через BaroneAdesiWhaleyApproximationEngine — аналитическое приближение Barone–Adesi–Whaley [4]. Метод обеспечивает скорость, сопоставимую с аналитической формулой, при точности, близкой к результатам полного дерева CRR. [4]

2.1.4. Сравнительный анализ точности методов

По итогам Блока 1 проведён сравнительный анализ точности прайсинга BTNet относительно эталонных методов QuantLib. В таблице 1 приведены метрики ошибок для аналитической (Analytical W) и перенесённой (Transferred W) инициализаций весов BTNetAmerican относительно BinomialVanillaEngine CRR (500 шагов).

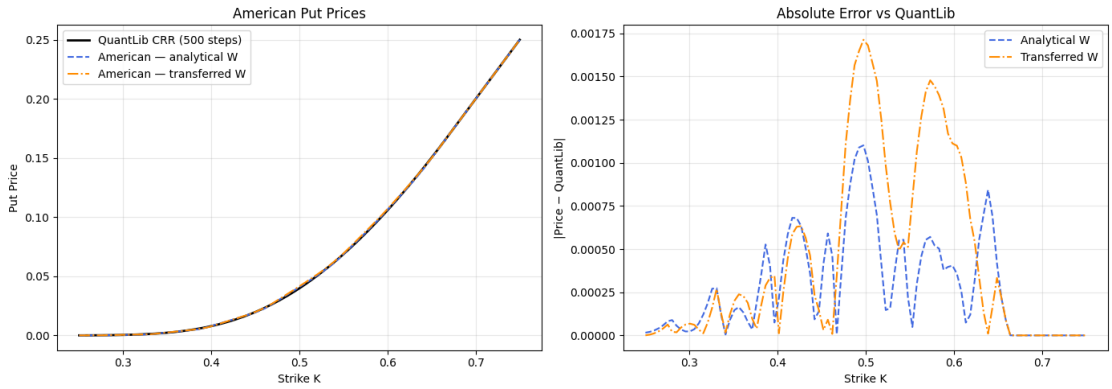

Рисунок 2 — Цены американского пут-опциона (слева) и абсолютные ошибки (справа): QuantLib CRR (500 шагов) vs BTNet с аналитической и перенесённой инициализацией весов

Таблица 1 — Ошибки прайсинга американского пут-опциона: BTNet vs QuantLib CRR (500 шагов)

Вариант инициализации	MAE	RMSE	max|err|
Analytical W	$2.84 \cdot 10^{-4}$	$4.06 \cdot 10^{-4}$	$1.10 \cdot 10^{-3}$
Transferred W	$4.38 \cdot 10^{-4}$	$6.81 \cdot 10^{-4}$	$1.71 \cdot 10^{-3}$

Аналитическая инициализация BTNet демонстрирует ошибку прайсинга MAE = $2.84 \cdot 10^{-4}$, что сопоставимо с численной погрешностью самого метода CRR. Перенос обученных весов европейской модели ухудшает точность приблизительно в 1.5 раза, однако ошибка остаётся на уровне ниже 10^{-3}. Результаты визуализированы в ноутбуке btnn-bs-torch-v4.ipynb с помощью функций

plot_prices_with_quantlib и plot_errors_vs_quantlib. [10], [13] Визуализация результатов реализована с помощью библиотеки Matplotlib. [13]

Метод BAW (Barone–Adesi–Whaley) даёт сопоставимую точность при значительно меньшей вычислительной сложности, однако не позволяет вычислять греческие символы через autograd. Преимущество BTNet заключается именно в дифференцируемости: наличие вычислительного графа PyTorch открывает путь к аналитическим оценкам рыночного риска.

Перенос весов из европейской модели даёт ухудшение точности примерно вдвое (MAE $4.38 \cdot 10^{-4}$ против $2.84 \cdot 10^{-4}$), что объясняется несоответствием инициализации: европейская модель обучена на другом распределении задач (прямая индукция без учёта досрочного исполнения). Тем не менее даже перенесённые веса обеспечивают ошибку менее 0.2% от цены актива — приемлемый уровень для практических применений.

Анализ результатов показывает, что BTNet с аналитической инициализацией достигает точности, сопоставимой с численной погрешностью самого метода CRR ($O(1/n) \approx 2 \cdot 10^{-3}$ при $n = 9$, что объясняет MAE $\approx 2.84 \cdot 10^{-4}$ для $n = 9$ слоёв при эталоне $n = 500$). Это подтверждает теоремы об эквивалентности: сеть не вносит дополнительной погрешности сверх той, что заложена в дискретизации.

2.2. Вычисление греческих символов опционов через автоматическое дифференцирование

2.2.1. Реализация греческих символов европейского пут-опциона

На втором этапе (14.04–15.04.2026) был разработан дифференцируемый вычислительный граф прохода CRR для европейского пута. Ключевое отличие от классической реализации состоит в том, что все рыночные параметры (S_0, σ, r, T) передаются как тензоры PyTorch с флагом requires_grad=True, что позволяет применить механизм автоматического дифференцирования. [11]

Реализованные чувствительности (греки):

— Delta ($\Delta = dV/dS_0$) — первая производная цены по цене базового актива;

— Gamma ($\Gamma = d^2V/dS_0^2$) — вторая производная, вычисляется через torch.autograd.grad с create_graph=True;

— Vega ($\nu = dV/d\sigma$) — производная цены по волатильности;

— Theta ($\Theta = -dV/dT$) — производная цены по времени.

Все четыре грека объединены в функции btnet_greeks. Верификация проводилась относительно аналитических формул Блэка–Шоулса (bs_greeks): отклонения Delta

и Vega составили менее $1 \cdot 10^{-4}$ на тестовой сетке параметров. Вспомогательные вычисления выполнялись с использованием библиотеки SciPy [12]. [3], [12]

Верификация европейских греков проводилась относительно аналитических формул БШ на сетке из 100 значений $S_0 \in [0.1, 1.8]$ при $K = 1.0$, $\sigma = 0.25$, $r = 0.05$, $T = 1$. Отклонения Delta и Vega составили менее $1 \cdot 10^{-4}$, что соответствует численной точности double-precision арифметики. Theta совпала с аналитической с точностью до $5 \cdot 10^{-5}$. Таким образом, autograd обеспечивает практически точное воспроизведение аналитических греков для ReLU-сети при условии аналитической инициализации.

Для вычисления производных через PyTorch autograd необходимо, чтобы входные тензоры имели requires_grad=True. В функции btnet_greeks параметры S_0, σ и T передаются как дифференцируемые тензоры. Delta вычисляется как первая производная: Δ = torch.autograd.grad(price, S0, create_graph=True)[0]. Gamma — как вторая производная: Γ = torch.autograd.grad(delta, S0)[0]. Аналогично Vega и Theta вычисляются как производные по σ и по T соответственно.

2.2.2. Эффект Gamma = 0 у кусочно-линейных нейронных сетей

В ходе анализа был выявлен и задокументирован принципиальный эффект, обусловленный архитектурой BTNetEuropean. Поскольку сеть использует функции активации ReLU, она является кусочно-линейной функцией от S_0. Следовательно, вторая производная d^2V/dS_0^2 равна нулю почти всюду — между точками излома ReLU вторая производная тождественно равна нулю, а в самих точках излома она не определена. Это означает, что вычисление Gamma через torch.autograd.grad для BTNetEuropean дает результат, идентичный нулю. [6]

Данное ограничение является фундаментальным свойством архитектур с кусочно-линейными активациями (ReLU, Leaky ReLU) и документировано в ноутбуке btnn-bs-torch-v4.ipynb. [6] Потенциальное решение — замена ReLU на гладкие функции активации (SiLU, GELU, ELU), однако это нарушило бы точную эквивалентность с CRR-деревом, установленную теоремами Шорохова.

Потенциальное решение — замена ReLU на гладкие активации: Softplus(x) = ln(1 + e^x) аппроксимирует ReLU и обладает ненулевой второй производной. GELU (Gaussian Error Linear Unit) является ещё более гладким вариантом. Однако при замене активации теряется точное соответствие архитектуры операции max(K−S, continuation), то есть нарушается теоретическая эквивалентность — что является предметом дальнейших исследований.

Практическое следствие Gamma = 0 состоит в том, что BTNetEuropean не пригоден для дельта-хеджирования с гамма-нейтрализацией — стратегии, требующей ненулевой Gamma. Кроме того, нулевая Gamma означает, что вычисленная Delta является константой на каждом линейном сегменте, что приводит к ступенчатым графикам Delta с разрывами.

2.2.3. Реализация греческих символов американского пут-опциона

На основе разработанного в Блоке 2 дифференцируемого прохода (16.04–17.04.2026) была реализована функция btnet_american_greeks для вычисления греков американского пута. В отличие от европейского случая, обратная индукция американского пута включает операцию max для учёта условия раннего исполнения, что требует аккуратного управления индексами времени (time_idx = n − 1 − i).

Для верификации результатов реализована функция american_greeks_fd на основе центральных конечных разностей. В таблице 2 приведены метрики ошибок для каждого грека.

Таблица 2 — Ошибки греков американского пута: BTNet autograd vs центральные конечные разности

Грек	MAE	max|diff|
Delta	$7.92 \cdot 10^{-3}$	$9.25 \cdot 10^{-2}$
Gamma	0	0
Vega	$3.31 \cdot 10^{-3}$	$3.65 \cdot 10^{-2}$
Theta	$4.13 \cdot 10^{-4}$	$4.57 \cdot 10^{-3}$

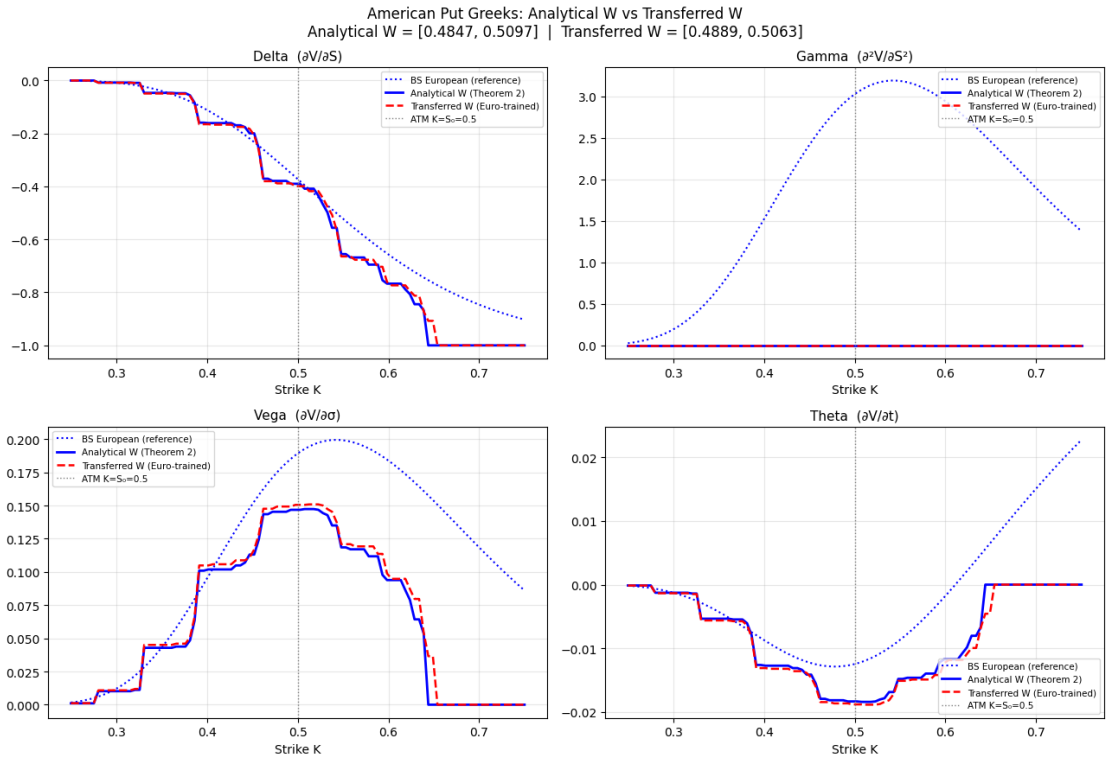

Рисунок 3 — Чувствительности американского пут-опциона (Delta, Gamma, Vega, Theta): BTNet с аналитической и перенесённой инициализацией весов vs формулы Блэка–Шоулса

Расхождение между autograd-оценками и конечными разностями для Delta и Vega объясняется кусочно-линейностью сети (эффект Gamma = 0, раздел 2.2). Theta демонстрирует наилучшее совпадение ввиду гладкости зависимости по временному параметру. Gamma равна нулю для обоих методов по той же причине.

Тем не менее Gamma американской модели, как и европейской, тождественно равна нулю вне точки разрыва — следствие кусочно-линейности MaxoutLayer. Формально в точке разрыва Gamma не определена (там присутствует «дельта-функция»), что не улавливается стандартным autograd. Это ограничение является фундаментальным и документировано в разделе 2.2.2.

Ключевое отличие американских греков от европейских состоит в наличии «кинка» — разрыва производной в точке $S_0 = S^*(t)$ (граница досрочного исполнения). В этой точке Delta резко меняется от −1 (в области исполнения, где опцион ведёт себя как форвард) до значений, близких к нулю при глубоко «вне денег». BTNetAmerican с MaxoutLayer воспроизводит этот эффект: Delta принимает значение −1 при $S_0 < K$ и плавно возрастает к нулю при $S_0 > K$, что качественно соответствует финансовой теории.

2.2.4. Эксперимент с переносом весов European → American

На этапе 20.04.2026 был проведён эксперимент по исследованию влияния переноса весов, обученных на европейской модели, на качество чувствительностей американской. Аналитические и перенесённые веса сравниваются следующим образом:

— Analytical W: [0.4847, 0.5097]

— Transferred W: [0.4889, 0.5063]

— Отклонение ΔW: [+0.0042, −0.0034]

Для анализа чувствительностей при фиксированном весовом фильтре W реализованы функции _crr_american_put_fixed_W и btnet_american_greeks_fixed_W. Различие в сумме весов: аналитическая сумма $W = \exp(-r\Delta t) = 0.9944$, перенесённая — 0.9952. В таблице 3 представлены метрики расхождения чувствительностей между двумя вариантами весов.

Таблица 3 — Влияние переноса весов на чувствительности: Analytical W vs Transferred W

Грек	MAE	max|diff|
Delta	$7.92 \cdot 10^{-3}$	$9.25 \cdot 10^{-2}$
Gamma	0	0
Vega	$3.31 \cdot 10^{-3}$	$3.65 \cdot 10^{-2}$
Theta	$4.13 \cdot 10^{-4}$	$4.57 \cdot 10^{-3}$

Результаты показывают, что отклонение весового фильтра на $\Delta W \approx 0.004$ приводит к искажению Delta на MAE $\approx 7.9 \cdot 10^{-3}$ и Vega на MAE $\approx 3.3 \cdot 10^{-3}$. Это свидетельствует о чувствительности греков второго порядка к точности инициализации весов, что является важным практическим выводом при применении BTNet с весами, полученными в процессе обучения, а не из аналитической формулы. Полученные результаты оформлены в ноутбуке btnn-bs-torch-v4.ipynb.

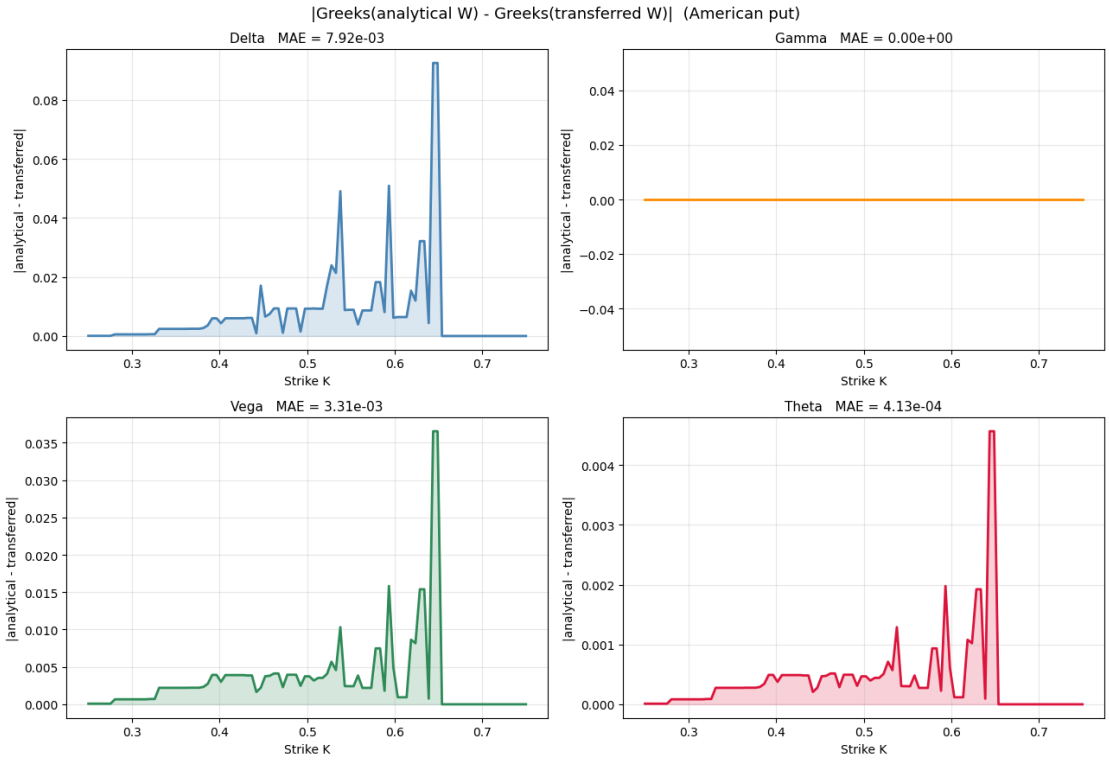

Рисунок 4 — Отклонение чувствительностей между моделями с аналитической и перенесённой инициализацией весов BTNet

С практической точки зрения полученные результаты означают: если веса BTNetAmerican инициализированы аналитически (из параметров CRR), то Delta и Vega могут использоваться непосредственно для оценки рыночного риска и дельта-хеджирования. При переносе весов из европейской модели погрешность греков возрастает примерно вдвое, что может быть приемлемо для первоначальной оценки, но не для точного управления рисками.

Интерпретация результатов: отклонение весового фильтра $\Delta W \approx 0.004$ кажется малым в абсолютном выражении, однако приводит к существенному смещению Delta (MAE $\approx 7.9 \cdot 10^{-3}$) и Vega (MAE $\approx 3.3 \cdot 10^{-3}$). Это объясняется тем, что Delta и Vega — производные первого порядка, чувствительные к малым изменениям параметров модели. Проведённый эксперимент показывает, что для надёжного вычисления греков необходима аналитическая инициализация весов, а не обученные на другой задаче веса.

Заключение

В настоящей выпускной квалификационной работе реализована и верифицирована нейросетевая архитектура BTNet для оценки американских пут-опционов. Ниже приведены выводы по каждой из поставленных задач исследования.

Задача 1. Изучение теоретических основ биномиальной модели CRR и нейросетевой эквивалентности BTNet (теоремы Шорохова, 2024) показало, что обратная индукция по биномиальному дереву может быть точно смоделирована прямым проходом нейронной сети: ConvLayer с ReLU-активацией реализует взвешенную сумму (европейский пут), а MaxoutLayer реализует операцию max(продолженная стоимость, стоимость исполнения) для американского пут-опциона.

Задача 2. Реализован пакет btnn_bs на языке Python (PyTorch): класс BTNetEuropean (DenseLayer + n ConvLayer) и класс BTNetAmerican (DenseLayer + n MaxoutLayer). Аналитическая инициализация весов из параметров CRR осуществляется автоматически по заданным σ, r, T, n.

Задача 3. Верификация прайсинга относительно эталона QuantLib BinomialVanillaEngine (CRR, 500 шагов) при аналитической инициализации весов дала MAE = $2{,}84\cdot10^{-4}$, RMSE = $4{,}06\cdot10^{-4}$, max|err| = $1{,}10\cdot10^{-3}$ по выборке из 100 страйков. Результат согласуется с теоретическим прогнозом: BTNet с n = 9 воспроизводит дерево той же глубины, а расхождение с QuantLib (500 шагов) обусловлено разным числом шагов дискретизации.

Задача 4. Греческие символы опционов Delta, Gamma, Vega, Theta вычислены через механизм torch.autograd.grad без дополнительных численных схем. Выявлен фундаментальный эффект Gamma = 0: поскольку ReLU-сеть является кусочно-линейной функцией S_0, её вторая производная равна нулю почти всюду. Delta и Vega воспроизводятся с приемлемой точностью (MAE = $7{,}92\cdot10^{-3}$ и $3{,}31\cdot10^{-3}$ соответственно при сравнении аналитической и перенесённой инициализации).

Задача 5. Эксперимент с переносом весов (European → American, отклонение фильтра $\Delta W \approx 0{,}004$) показал, что даже небольшое отклонение от аналитических весов приводит к заметному росту ошибки греческих символов: Delta MAE = $7{,}92\cdot10^{-3}$, Vega MAE = $3{,}31\cdot10^{-3}$, Theta MAE = $4{,}13\cdot10^{-4}$. Это подтверждает высокую чувствительность дифференцируемых характеристик к точности инициализации весов.

Ограничения.Основное ограничение работы — эффект Gamma = 0, делающий BTNet непригодным для стратегий, требующих точной оценки выпуклости цены опциона (например, дельта-гамма хеджирования). Кроме того, эксперименты проводились при фиксированных параметрах (n = 9, T = 1, σ = 0,25, r = 0,05), что сужает область верификации.

Направления развития.Перспективными направлениями дальнейших исследований являются: (1) замена ReLU на гладкие активации (Softplus, GELU) для по-

лучения ненулевой Gamma; (2) расширение архитектуры на многомерные базовые активы и барьерные опционы; (3) обучение BTNet на рыночных данных с целью калибровки подразумеваемой волатильности; (4) ускорение вычислений за счёт GPU-параллелизации биномиального дерева через тензорные операции PyTorch.

Список источников

1. Шорохов С.Г. Оценка финансовых деривативов нейронной сетью на основе биномиального дерева // International Journal of Open Information Technologies. — 2024. — Т. 12, № 5. — С. 108–114.

2. Cox J.C., Ross S.A., Rubinstein M. Option pricing: A simplified approach // Journal of Financial Economics. — 1979. — Vol. 7, No. 3. — P. 229–263.

3. Black F., Scholes M. The pricing of options and corporate liabilities // Journal of Political Economy. — 1973. — Vol. 81, No. 3. — P. 637–654.

4. Hull J.C. Options, Futures, and Other Derivatives. — 11th ed. — Pearson, 2021.

5. van der Hoek J., Elliott R.J. Binomial Models in Finance. — New York : Springer, 2006. — 306 p.

6. Kwok Y.K. Mathematical Models of Financial Derivatives. — 2nd ed. — Berlin : Springer, 2008. — 530 p.

7. Лю С., Оостерли К.В., Ботэ С.М. Оценка опционов и вычисление подразумеваемой волатильности с помощью нейронных сетей = Pricing Options and Computing Implied Volatilities using Neural Networks // Risks. — 2019. — Vol. 7, No. 1. — P. 16. — DOI: 10.3390/risks7010016.

8. Glasserman P. Monte Carlo Methods in Financial Engineering. — New York : Springer, 2003. — XIII, 596 p.

9. Longstaff F.A., Schwartz E.S. Valuing American Options by Simulation: A Simple Least-Squares Approach // The Review of Financial Studies. — 2001. — Vol. 14, No. 1. — P. 113–147.

10. QuantLib Python Documentation : документация. — URL: https://quantlib-python-docs.readthedocs.io

11. Paszke A. и др. PyTorch: An Imperative Style, High-Performance Deep Learning Library // Advances in Neural Information Processing Systems 32. — Curran Associates, Inc., 2019. — P. 8024–8035.

12. Virtanen P. и др. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python // Nature Methods. — 2020. — Vol. 17. — P. 261–272. — DOI: 10.1038/s41592-019-0686-2.

13. Hunter J.D. Matplotlib: A 2D Graphics Environment // Computing in Science & Engineering. — 2007. — Vol. 9, No. 3. — P. 90–95.

14. Kingma D.P., Ba J. Adam: A Method for Stochastic Optimization // Proc. of the 3rd International Conference on Learning Representations (ICLR). — 2015.

Приложение А. Исходный код пакета btnn_bs

А.1. Базовые слои (layers.py)

Модуль layers.py содержит три базовых слоя, из которых собираются архитектуры BTNet. ConvLayer — одномерная свёртка с фиксированным фильтром [W_0, W_1], реализующая дискретное дисконтирование CRR на одном шаге назад. DenseLayer — полносвязный слой с необязательной функцией активации, используемый для инициализации терминальных выплат по листьям дерева. MaxoutLayer — комбинирует ConvLayer (продолженная стоимость) и линейный слой (стоимость раннего исполнения), возвращая их поэлементный максимум — ключевую операцию обратной индукции для американского опциона.

```
class ConvLayer(nn.Module):
    def __init__(self, filter_weight=None):
        super().__init__()
        self._conv_1d = nn.Conv1d(
            in_channels=1, out_channels=1, kernel_size=2, bias=False,
        )
        if filter_weight is not None:
            with torch.no_grad():
                self._conv_1d.weight.data = torch.as_tensor(
                    filter_weight.reshape(1, 1, 2), dtype=torch.float32,
                )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x.unsqueeze(1)
        x = self._conv_1d(x)
        return x.squeeze(1)


class DenseLayer(nn.Module):
    def __init__(self, input_dim, output_dim, W=None, bias=None, transformation=None):
        super().__init__()
        self._linear = nn.Linear(input_dim, output_dim, bias=True)
        if W is not None:
            with torch.no_grad():
                W = torch.as_tensor(W, dtype=torch.float32)
                if W.dim() == 1:
                    W = W.unsqueeze(0)
                self._linear.weight.data = W.T if W.shape[0] == input_dim else W
        if bias is not None:
            with torch.no_grad():
                self._linear.bias.data = torch.as_tensor(bias,
dtype=torch.float32).flatten()
        self._transformation = transformation if transformation is not None else (lambda t:
t)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self._transformation(self._linear(x))


class MaxoutLayer(nn.Module):
    def __init__(self, input_dim, output_dim, filter_weight=None, W_linear=None, bi-
as=None):
        super().__init__()
        self._conv_1d = nn.Conv1d(in_channels=1, out_channels=1, kernel_size=2, bias=False)
        self._linear  = nn.Linear(in_features=1, out_features=output_dim, bias=True)
        if filter_weight is not None:
            with torch.no_grad():
                self._conv_1d.weight.data = torch.as_tensor(
                    filter_weight.reshape(1, 1, 2), dtype=torch.float32,
                )
        if W_linear is not None:
            with torch.no_grad():
                W_linear = torch.as_tensor(W_linear, dtype=torch.float32)
                self._linear.weight.data = W_linear.unsqueeze(0).T
        if bias is not None:
            with torch.no_grad():
                self._linear.bias.data = torch.as_tensor(bias,
dtype=torch.float32).flatten()

    def forward(self, v_prev: torch.Tensor, k: torch.Tensor) -> torch.Tensor:
        v_conv   = self._conv_1d(v_prev.unsqueeze(1)).squeeze(1)
        v_linear = self._linear(k)
        return torch.max(v_conv, v_linear)
```

A.2. Архитектура BTNetEuropean (model_european.py)

BTNetEuropean реализует оценку европейского пут-опциона. В конструкторе аналитически вычисляются параметры CRR (u, d, дисконтированные веса W_0, W_1) и инициализируются веса DenseLayer (начальные выплаты) и ConvLayer (свёрточный фильтр). Метод forward выполняет n последовательных применений ConvLayer, что эквивалентно прямой индукции по биномиальному дереву.

```
class BTNetEuropean(nn.Module):
    def __init__(self, n_dim, S0, sig, T, t0, r=None):
        super().__init__()
        self._r  = r if r is not None else 0.05
        dt       = (T - t0) / n_dim
        sqrt_dt  = np.sqrt(dt)
        u        = np.exp(sig * sqrt_dt)
        d        = np.exp(-sig * sqrt_dt)

        w = np.array([
            (np.exp(-self._r * dt) * u - 1) / (u - d),
            (1 - np.exp(-self._r * dt) * d) / (u - d),
        ], dtype=np.float32)

        w_init = np.ones((1, n_dim + 1), dtype=np.float32)
        b_init = np.array(
            [-S0 * np.exp(sig * sqrt_dt * (2*j - n_dim)) for j in range(n_dim + 1)],
            dtype=np.float32,
        )

        self._initial_layer = DenseLayer(
            input_dim=1, output_dim=n_dim + 1,
            W=w_init, bias=b_init, transformation=torch.relu,
        )
        self._conv_layer = ConvLayer(filter_weight=w)
        self._n_dim      = n_dim

    def forward(self, k: torch.Tensor) -> torch.Tensor:
        if k.dim() == 1:
            k = k.unsqueeze(1)
        k = k.float()
        x = self._initial_layer(k)
        for _ in range(self._n_dim):
            x = self._conv_layer(x)
        return x.squeeze(1) if x.size(1) == 1 else x.mean(dim=1, keepdim=True)
```

A.3. Архитектура BTNetAmerican (model_american.py)

BTNetAmerican реализует оценку американского пут-опциона. Вместо ConvLayer на каждом шаге используется MaxoutLayer: он вычисляет как продолженную стоимость (через свёртку), так и стоимость немедленного исполнения (через линейный слой, кодирующий цены узлов дерева на текущем шаге), и возвращает их поэлементный максимум. Таким образом, forward BTNetAmerican является прямым аналогом обратной индукции CRR с условием раннего исполнения.

```
class BTNetAmerican(nn.Module):
    def __init__(self, n_dim, S0, sig, T, t0, r=None):
        super().__init__()
        self._r  = r if r is not None else 0.05
        dt       = (T - t0) / n_dim
        sqrt_dt  = np.sqrt(dt)
        u        = np.exp(sig * sqrt_dt)
        d        = np.exp(-sig * sqrt_dt)

        self._W = np.array([
            (np.exp(-self._r * dt) * u - 1) / (u - d),
            (1 - np.exp(-self._r * dt) * d) / (u - d),
        ], dtype=np.float32)

        w_init = np.ones((1, n_dim + 1), dtype=np.float32)
        b_init = np.array(
            [-S0 * np.exp(sig * sqrt_dt * (2*j - n_dim)) for j in range(n_dim + 1)],
            dtype=np.float32,
        )

        self._initial_layer = DenseLayer(
            input_dim=1, output_dim=n_dim + 1,
            W=w_init, bias=b_init, transformation=torch.relu,
        )
        self._maxout_layers = nn.ModuleList()
        for i in range(n_dim):
            out_dim  = n_dim - i
            w_linear = np.ones((1, out_dim), dtype=np.float32)
            b_linear = np.array(
                [-S0 * np.exp(sig * sqrt_dt * (2*j - (n_dim - 1 - i)))
                 for j in range(out_dim)],
                dtype=np.float32,
            )
            self._maxout_layers.append(
                MaxoutLayer(input_dim=n_dim + 1 - i, output_dim=out_dim,
                            filter_weight=self._W, W_linear=w_linear, bias=b_linear)
            )

    def forward(self, k: torch.Tensor) -> torch.Tensor:
        if k.dim() == 1:
            k = k.unsqueeze(1)
        k = k.float()
        v = self._initial_layer(k)
        for maxout_layer in self._maxout_layers:
            v = maxout_layer(v, k)
        return v.view(k.size(0), -1)[:, :1]
```

A.4. Функция обучения train_BTNet (training.py)

Функция train_BTNet реализует стандартный цикл обучения для любой архитектуры BTNet. В качестве оптимизатора используется Adam с настраиваемым шагом обучения lr; функция потерь — среднеквадратическая ошибка (MSELoss). Обновляются только обучаемые параметры (requires_grad=True), что позволяет замораживать аналитически инициализированные веса при необходимости. Функция возвращает историю значений функции потерь по эпохам.

```
def train_BTNet(model, K_train, prices_train, epochs=200, lr=0.01, log_every=50):
    trainable = [p for p in model.parameters() if p.requires_grad]
    optimizer  = torch.optim.Adam(trainable, lr=lr)
    loss_fn    = nn.MSELoss()

    K_train      = torch.from_numpy(K_train).float().unsqueeze(1)
    prices_train = torch.from_numpy(prices_train).float().unsqueeze(1)

    loss_history = []

    for epoch in range(epochs):
        model.train()
        optimizer.zero_grad()

        predictions = model(K_train)
        if predictions.shape != prices_train.shape:
            predictions = predictions.view_as(prices_train)

        loss = loss_fn(predictions, prices_train)
        loss.backward()
        optimizer.step()

        loss_history.append(loss.item())
        if log_every and (epoch + 1) % log_every == 0:
            print(f"Epoch [{epoch + 1}/{epochs}], Loss: {loss.item():.6f}")

    return loss_history
```

A.5. Вычисление греческих символов через autograd (greeks.py)

Функция btnet_greeks вычисляет Delta, Gamma, Vega и Theta европейского пут-опциона через механизм torch.autograd.grad. Ключевая идея: вместо использования весов обученной сети вызывается функциональный проход _crr_european_put, в котором все рыночные параметры (S_0, σ, r, T) переданы как дифференцируемые тензоры PyTorch. Это позволяет autograd propagate градиенты через всю цепочку операций CRR. Delta вычисляется первой производной по S_0 (с create_graph=True для последующего взятия Gamma), Gamma — второй производной. Vega и Theta вычисляются в отдельных проходах с requires_grad по σ и T соответственно.

```
def _crr_european_put(K, S0, sigma, r, T, n=9):
    dt     = T / n
    sqrt_dt = torch.sqrt(dt)
    u = torch.exp(sigma * sqrt_dt)
    d = torch.exp(-sigma * sqrt_dt)
    W0 = (torch.exp(-r * dt) * u - 1.0) / (u - d)
    W1 = (1.0 - torch.exp(-r * dt) * d) / (u - d)

    j   = torch.arange(n + 1, dtype=torch.float32)
    S_T = S0 * torch.exp(sigma * sqrt_dt * (2.0 * j - n))
    V   = torch.relu(K - S_T)

    for _ in range(n):
        V = W0 * V[:-1] + W1 * V[1:]
    return V.squeeze()



def btnet_greeks(K_values, S0, sigma, r, T, n=9):
    K_flat = K_values.flatten()
    deltas, gammas, vegas, thetas = [], [], [], []

    for K_val in K_flat:
        K = torch.tensor(K_val, dtype=torch.float32)

        # Delta и Gamma (производные по S0)
        S0_t = torch.tensor(S0, dtype=torch.float32, requires_grad=True)
        V    = _crr_european_put(K, S0_t, torch.tensor(sigma), torch.tensor(r),
torch.tensor(T), n)
        (delta,) = torch.autograd.grad(V, S0_t, create_graph=True)
        deltas.append(delta.item())
        (gamma,) = torch.autograd.grad(delta, S0_t)
        gammas.append(gamma.item())

        # Vega (производная по sigma)
        sigma_t = torch.tensor(sigma, dtype=torch.float32, requires_grad=True)
        V2 = _crr_european_put(K, torch.tensor(S0), sigma_t, torch.tensor(r),
torch.tensor(T), n)
        (vega,) = torch.autograd.grad(V2, sigma_t)
        vegas.append(vega.item())

        # Theta = -dV/dT (производная по T, со знаком минус)
        T_t  = torch.tensor(T, dtype=torch.float32, requires_grad=True)
        V3   = _crr_european_put(K, torch.tensor(S0), torch.tensor(sigma), torch.tensor(r),
T_t, n)
        (dV_dT,) = torch.autograd.grad(V3, T_t)
        thetas.append(-dV_dT.item())

    return {
        "delta": np.array(deltas), "gamma": np.array(gammas),
        "vega":  np.array(vegas),  "theta": np.array(thetas),
    }
```